Personal Finance Manager

Iteration 2 — Written Deliverable

CSE 3311: Object-Oriented Software Engineering
Team 6
Ujjwal Sood • Hardik Kari • David Aviles • Ishika Sanghadia
Instructor: Christoph Csallner

Executive Summary

Iteration 2 moves the Personal Finance Manager from a mainly manual finance tracker toward connected-bank data and automated insight. The iteration adds a reported Plaid service and API layer, persistence models for bank links, a Bank Sync interface, subscription detection, data-driven notifications, and interface improvements. The frontend production build and selected automated tests passed. The release remains an internal, partially verified build because a Plaid sandbox flow was not demonstrated, and the complete backend suite was blocked during collection by an environment dependency problem.

Table 1. Iteration 2 status at a glance

Workstream	Evidence-based status	Release interpretation
Plaid bank synchronization	Implemented in service/API/UI code; sandbox flow not verified	Partial
Subscription insights	Three new domain tests reported passing	Complete at code/test level
Notifications	Hardcoded examples replaced with budget and recurring-payment data	Complete at code level
UI and accessibility styling	Production build passed; browser and accessibility audits not recorded	Complete at build level
Backend verification	Seven executable tests passed; full suite stopped on five import errors	Blocked / incomplete

Evidence rule used in this report: “implemented” describes supplied code evidence; “verified” is used only when a recorded test or demonstration result exists. This distinction prevents unfinished verification from being presented as completed functionality.

1. Planning

1.1 Baseline and iteration objective

Iteration 1 established manual income/expense tracking and a purchase affordability check. Iteration 2 prioritizes bank synchronization because connected transaction data reduces repeated manual entry and creates the data foundation needed for subscription summaries and relevant notifications.

1.2 MoSCoW priorities and rationale

Table 2. MoSCoW feature priorities

Priority	Feature	Why it belongs in this category	Risk response
Must	Bank connection, secure token storage, incremental sync, duplicate prevention	Without reliable transaction ingestion, the iteration’s automated insights have no dependable data source.	Mitigates data-entry burden; requires sandbox and regression verification.
Must	Subscription detection and summaries	Provides immediate user value from synchronized history and tests a core automated-insight capability.	Tests should cover known merchants, stable monthly charges, and false positives.
Must	Data-driven notifications	Removes misleading hardcoded alerts and ties messages to actual budgets and recurring activity.	Reduces incorrect UI content; requires runtime validation.
Should	Responsive, accessible status and error states	Users must understand connection, synchronization, and failure states across common screen sizes.	Reduces usability and recovery risk.

Priority	Feature	Why it belongs in this category	Risk response
Could	Additional visual refinement and advanced filters	Useful after the core flow is stable, but does not determine sync correctness.	Defer when verification is at risk.
Won't this iteration	Editable commitments, transaction correction, separate reserve displays	An early UI change dismissed these previous priorities	Record as deferred rather than silently presenting them as complete.

1.3 Work breakdown, resources, and schedule

Table 3. Iteration work packages

Phase	Task and output	Primary evidence	Status
Analysis	Confirm user need, scope, acceptance criteria, and deferred Iteration 1 features	Prioritized requirement set	Incomplete record
Design	Define service/API/UI responsibilities, persistence links, and exceptional flows	Figures 1–3; Tables 4–6	Documented provisionally
Code	Implement Plaid integration, subscription logic, alerts, and UI states	E1–E5	Reported complete/partial by feature
Unit test	Test subscription rules and backend services	E6 and backend tests	Partial
Integrate	Connect UI, API, service, provider, and database	Integration checklist	Partial
System test	Run end-to-end sandbox, restart, responsive, and accessibility checks	System test record	Not completed
Release	Tag a reproducible internal build with setup instructions and known limits	Commit/tag and release checklist	Not ready

The course has three project iterations; this report treats Iteration 2 as integration and risk reduction, while Iteration 3 should focus on completing deferred user features, full system verification, and release quality.

1.4 Competitor-analysis decision

Competitor analysis should be updated when the product direction changes. Adding bank synchronization and subscription insight places the app closer to mature finance tools, where synchronization is an expected baseline rather than a unique differentiator. The useful planning conclusion is that Iteration 3 should differentiate through transparent calculations, clear confidence/limitations for detected subscriptions, and focused decision support such as "Can I Afford It?"

2. Requirements

2.1 Users and system boundary

The primary user is an authenticated person who wants a consolidated view of personal finances. External actors are the Plaid service, the application database, and the browser. The application is responsible for authentication, account/transaction persistence, synchronization decisions, insight calculations, and user-visible status. Plaid supplies institution and transaction data; it does not replace application validation or error handling.

Table 4. Functional and quality requirements

ID	Requirement	Inputs / data	Expected output	Priority
FR-01	The system shall let an authenticated user start and complete a bank connection.	User identity; Link/public token	Stored connection and visible status	Must
FR-02	The system shall synchronize added, modified, and removed provider transactions.	Connection cursor; provider transaction payloads	Updated local transactions and cursor	Must
FR-03	The system shall prevent duplicate imports by provider transaction identifier.	Provider transaction ID	At most one corresponding local transaction	Must
FR-04	The system shall reconcile imported transactions with account balance data.	Imported amounts; institution balance	Consistent account balance/baseline	Must
FR-05	The system shall identify likely subscriptions and label totals as estimates.	Transaction merchant, amount, and dates	Count, names, latest charges, estimated monthly total	Must
FR-06	The system shall generate notifications from stored budgets and recurring payments.	Budget and recurring-payment records	Relevant notifications with retained read state	Must
FR-07	The user shall be able to refresh, disconnect, and inspect connection status.	Connection action	Updated state or actionable error	Should
NFR-01	Access tokens shall be encrypted at rest and excluded from client-visible responses.	Provider access token	Encrypted server-side value	Must
NFR-02	Interactive controls shall have keyboard focus, disabled, status, and reduced-motion behavior.	Keyboard/pointer and display settings	Understandable responsive interaction	Should

2.2 Exceptional and boundary conditions

The acceptance criteria must cover invalid or expired connection tokens, provider/network failure, repeated synchronization, pending-to-posted transitions, deleted transactions, disconnect behavior, negative starting baselines, no detectable subscriptions, and empty notification states. These conditions are requirements because they change what users see and whether stored financial data remains trustworthy.

3. Design

3.1 Architecture and responsibilities

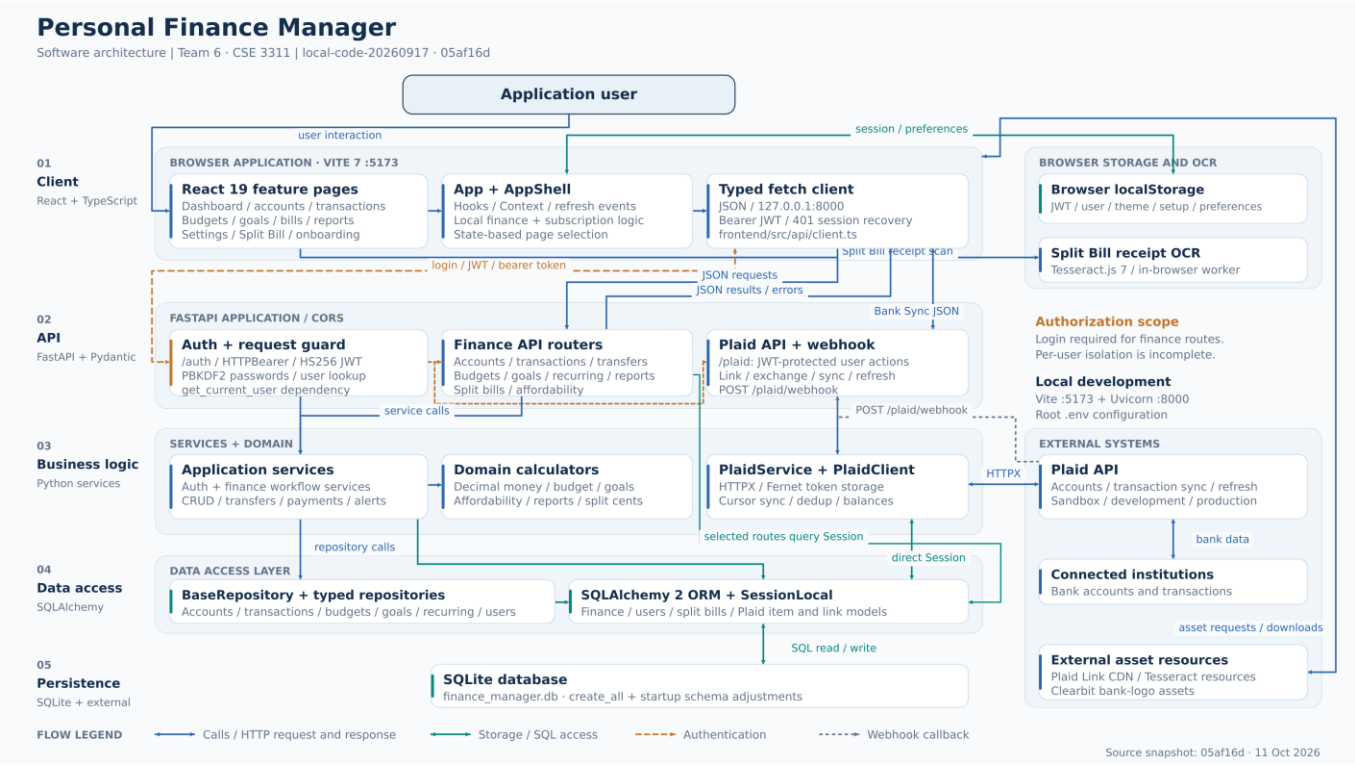


Figure 1. Iteration 2 deployment and data-flow architecture.

The React/Vite client sends authenticated HTTP/JSON requests to FastAPI routes. The routes perform authorization and validation, then call application services. Plaid-specific service code communicates with the Plaid sandbox/API and owns synchronization, duplicate prevention, and balance reconciliation. SQLAlchemy models persist users, accounts, transactions, encrypted token data, and provider link identifiers in SQLite. Subscription and notification logic consumes the stored finance data and returns derived results to the client. Plaid does not communicate directly with the browser except through the provider-managed Link flow initiated by the application.

3.2 Use-case model

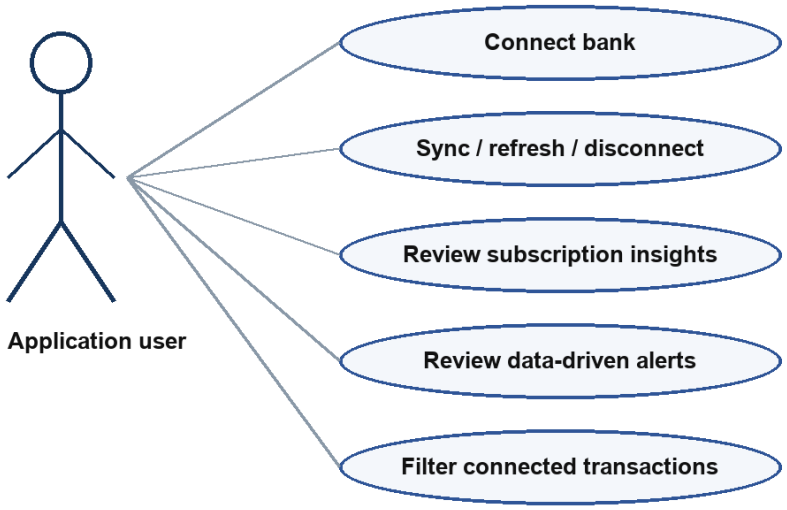


Figure 2. Iteration 2 user-facing use cases.

Table 5. Use-case summaries

Use case	Normal outcome	Important exception
Connect bank	Connection record is created and status becomes visible.	Token exchange or institution connection fails; show actionable error and store no unusable credential.
Sync / refresh / disconnect	Local accounts and transactions reflect provider changes.	Repeated sync, removed item, webhook failure, or disconnect must not duplicate/corrupt data.
Review subscription insights	Likely recurring charges and an estimated monthly total are displayed.	No candidates or uncertain candidates must be explained rather than shown as confirmed bills.
Review notifications	Alerts reflect actual budget and recurring-payment data.	Empty/stale data should not produce fabricated notices.
Filter transactions	User distinguishes connected-bank records from other sources.	Empty result preserves context and offers a clear recovery path.

3.3 Physical data model

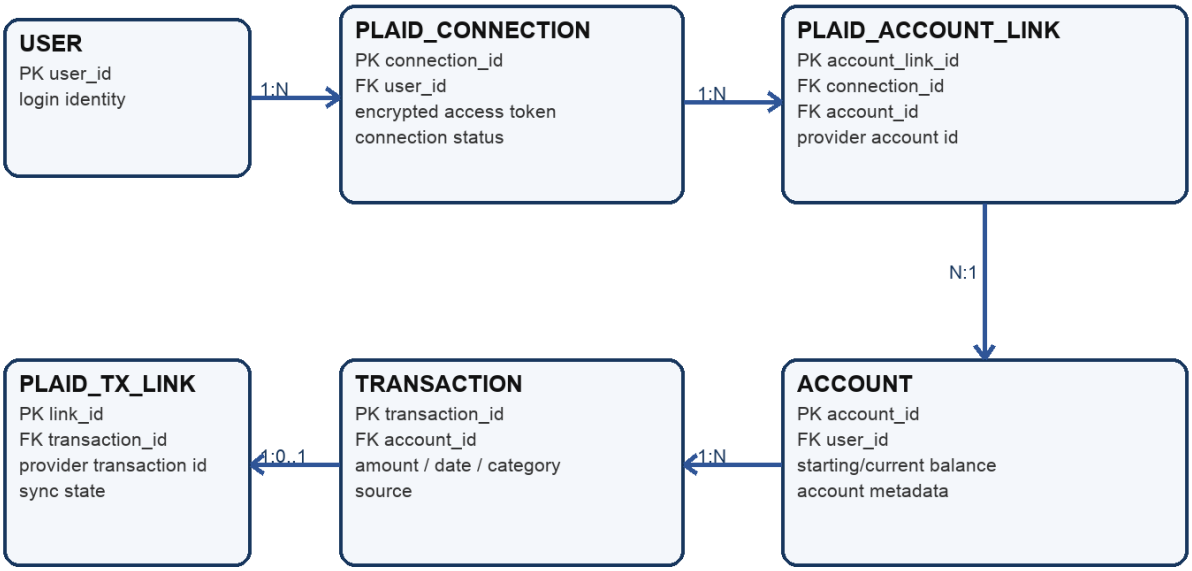


Figure 3. Provisional physical persistence model

Figure 3 records the physical relationships needed by the reported behavior: a user owns connections and accounts; a connection maps provider accounts to local accounts; accounts own transactions; provider transaction identifiers link imports to local records for deduplication.

3.4 Key design decisions

Table 6. Design decisions and consequences

Decision	Reason	Consequence / validation needed
Encrypt provider access tokens on the server	Credentials must not be exposed to the browser or stored as plaintext.	Verify encryption key configuration, rotation assumptions, and response schemas.
Keep provider IDs in link records	Stable external identifiers support deduplication and update/removal handling.	Test repeated sync and pending-to-posted transitions.
Use a domain module for subscription detection	Rules can be reused during imports and tested without UI rendering.	Document thresholds and false-positive behavior.
Preserve read/unread state during refresh	Refreshing derived data should not erase user interaction state.	Add notification-specific regression tests.

4. Code and Implementation

Table 7. Feature-to-code evidence map

Evidence	Responsibility	Implementation status	Verification status
E1 — backend/app/services/plaid_service.py	Link token, token exchange, encrypted access token, incremental sync, duplicate prevention, reconciliation	Reported implemented	Sandbox and blocked backend tests remain
E2 — backend/app/api/plaid.py	Connection actions, status, sync/refresh/disconnect, webhook verification and update handling	Reported implemented	Exact routes/schemas require final code review

Evidence	Responsibility	Implementation status	Verification status
E3 — frontend/src/components/PlaidConnection.tsx	Bank Sync UI and connection-state polling	Reported implemented	Production build passed; live flow unverified
E4 — frontend/src/domain/subscriptionInsights.ts	Known-merchant and stable monthly-charge detection; summaries	Implemented	Three new tests reported passed
E5 — frontend/src/components/AppShell.tsx	Data-driven notifications and read/unread state preservation	Implemented	No dedicated runtime test supplied
Database link models	Connection, account-link, and transaction-link persistence	Reported implemented	Exact model/migration evidence needed

4.1 Synchronization algorithm

The distinctive implementation is incremental synchronization. The service reads the next provider batch, applies added records, updates modified records, removes deleted records, and advances the synchronization cursor. Provider transaction IDs are the idempotency key: a repeated import should update or ignore an existing mapping rather than create a second local transaction. Reconciliation may compute a starting baseline from the institution-reported current balance and imported transaction history, including a negative baseline when the numbers require it. This rule must be verified with fixed examples because a balance error can propagate into budgets and affordability results.

4.2 Subscription-detection algorithm

Subscription candidates come from recognized recurring merchants and from sufficiently stable monthly charges for otherwise unknown merchants. The UI reports a count, names, latest charges, linked accounts, and an estimated monthly total. The word “estimated” is necessary because pattern detection cannot prove that a charge is a current contractual obligation.

5. Unit Testing

The test record below uses standard input, expected-output, and result fields.

Table 8. Unit and component test cases

ID / component	Input or setup	Expected output	Reported result
UT-SUB-01 / known merchant	Monthly transactions from a recognized subscription merchant	One candidate with latest charge and estimated monthly amount	Passed within 3-test subscription set; exact fixture needed
UT-SUB-02 / stable unknown merchant	Repeated monthly charges from an unknown merchant within configured tolerances	One inferred subscription candidate	Passed within 3-test subscription set; exact fixture needed
UT-SUB-03 / nonrecurring charges	Irregular dates or materially different amounts	No false subscription candidate	Passed within 3-test subscription set; exact fixture needed
UT-PLAID-01 / deduplication	Run the same provider transaction batch twice	Only one local transaction per provider ID	Test reportedly exists; execution not established
UT-BAL-01 / negative baseline	Current balance lower than net imported history	Negative starting baseline while current balance remains correct	Test exists; blocked by environment
UT-NOTIFY-01 / refresh state	Budget/recurring records plus read notification state	Derived notices refresh while read state persists	Needed

Table 9. Recorded automated verification

Verification	Result	Meaning
Frontend automated tests	16 passed, 0 failed: 8 auth/session, 5 Split Bill, 3 subscription insight	Only the three subscription tests are explicitly new; remaining tests provide regression coverage.
Frontend production build	Passed	Confirms compilation/bundling; does not prove browser, device, or accessibility behavior.
Executable backend subset	7 passed	Scope and test names must be recorded.
Full backend suite	Not completed; collection stopped on 5 pydantic_core import errors	Environment blocker; no claim of a fully passing backend suite.
Plaid and balance tests	Present but not executed in the inspected environment	Rerun after rebuilding the environment.

6. Integration

Integration joins five boundaries: authenticated browser to FastAPI, FastAPI to the Plaid service, service to the provider, service to the database, and stored data back to subscription and notification views. The frontend build verifies static compatibility, but end-to-end bank synchronization remains unverified.

Table 10. Integration checklist

Boundary	Acceptance condition	Current evidence
Browser → API	JWT-protected request returns status without exposing provider access token	Authentication support exists; exact Plaid response inspection needed
API → service	Each action validates input and maps service errors into stable responses	Reported code exists; route/schema review needed
Service → Plaid	Link, exchange, sync, refresh, disconnect, and webhook paths work in sandbox	Not demonstrated
Service → database	Connections and links survive restart; repeated sync remains idempotent	Models reported; restart and repeated-sync evidence missing
Stored data → insights	Imported transactions feed subscription and notification logic	Design intent reported; end-to-end demonstration missing

7. System Testing

Table 11. Required system test script

Step	Action	Expected observable result	Status
1	Create/login to an account and open Account Settings	Authenticated session and Bank Sync panel visible	Previously working; rerun on release build
2	Connect a Plaid sandbox institution	Connection succeeds without exposing credentials	Not verified
3	Run initial sync	Accounts and transactions appear once with source labels	Not verified

Step	Action	Expected observable result	Status
4	Run sync again, then modify/remove provider data	No duplicate; modifications/removals are reflected	Not verified
5	Restart backend and frontend	Connection and financial data remain available	Not verified
6	Open subscription insights	Count, candidates, latest charges, accounts, and estimated total are consistent	Not verified end to end
7	Trigger budget/recurring conditions and refresh notifications	Only data-driven alerts appear; read state persists	Not verified
8	Test narrow/wide screens, keyboard focus, reduced motion, empty and error states	Layout remains usable and status messages are announced/clear	Not verified

8. Internal Release

8.1 Setup and reproducibility

Setup work includes root .env loading, an .env.example file, Plaid credential/webhook instructions, and required HTTP, JWT, and cryptography dependencies in backend runtime dependencies. Before the final release, we must verify exact environment-variable names, dependency versions, database migration steps, backend/frontend launch commands, and test commands in a clean environment.

Table 12. Internal release gate

Gate	Required evidence	Current decision
Versioned baseline	Commit or tag containing all Iteration 2 work	Not ready — current changes reported uncommitted
Clean installation	Documented setup succeeds on a clean machine	Not verified
Automated regression	Full frontend and backend suites execute and pass or have accepted known failures	Not ready — backend collection blocked
Sandbox acceptance	Connect/sync/repeat/update/remove/disconnect paths recorded	Not verified
Persistence	Restart test recorded	Not verified
Known limitations	Deferred features and unresolved risks listed	Documented in this report

Repository: https://github.com/soodujjwal/Team_6_Object_Oriented_Software.

9. Users and Feedback

The table below shows an ideal session to receive meaningful feedback from users for this iteration

Table 13. Minimum feedback protocol

Task / question	Evidence to record	Decision supported
Connect a sandbox bank and locate synchronized transactions. What was unclear?	Completion, hesitation, error, short quotation	Connection flow and status wording

Task / question	Evidence to record	Decision supported
Review the subscription summary. Which result looks wrong or needs explanation?	Candidate judged correct/incorrect and reason	Detection thresholds and “estimated” labeling
Review notifications and choose the most useful one.	Priority choice and missing alert	Notification ordering/content
Use keyboard and narrow screen for one core task.	Observed navigation/layout issue	Accessibility and responsive fixes

An early follow-up with the same user from the previous iteration was able to be conducted and expressed better interest in the application due to the improved user interface. More specifically, the usage of the application on mobile devices.

10. Risk Reassessment

Table 14. Iteration 2 risk reassessment

Risk	Initial exposure	Current exposure	Response and next trigger
Incorrect bank synchronization	High	High	Mitigation code exists, but exposure remains high until sandbox added/modified/removed/repeated-sync cases pass.
Backend environment prevents verification	Medium	High	Recreate/pin the environment immediately; trigger is successful collection and full-suite execution.
Credential or token exposure	High	Medium	Encrypted storage is reported; verify configuration, responses, logs, disconnect, and secret handling.
Subscription false positives	Medium	Medium	Three rule tests reduce uncertainty; user review and edge-case tests remain.
Scope displacement of Iteration 1 work	Medium	High	Explicitly accept/defer earlier features and schedule them in Iteration 3.
Unreproducible release / weak attribution	Medium	High	Commit all work, tag cutoff, save logs, and map each task to an owner before submission.

11. Conclusion and Iteration 3 Priorities

Iteration 2 reports meaningful architectural progress: connected-bank components, reusable subscription logic, data-driven notifications, and a buildable frontend. The strongest verified result is the subscription test set; the largest unresolved risk is end-to-end bank synchronization and backend verification.

Table 15. Ordered next actions

Order	Action	Completion evidence
1	Rebuild/pin backend environment and run the full suite	Named command, commit, timestamp, pass/fail log
2	Run sandbox connect/sync/repeat/update/remove/disconnect scenarios	Completed system-test record with actual results
3	Verify database migration and restart persistence	Before/after restart record
4	Add notification, deduplication, webhook, and reconciliation regression tests	Passing tests tied to requirements

Order	Action	Completion evidence
5	Conduct and document a full user session	Participants, tasks, observations, decisions
6	Resolve deferred Iteration 1 features and finalize Iteration 3 plan	Accepted priorities, owners, effort, and dates
7	Commit, tag, attribute, and package the internal release	Release commit/tag and contribution record

References and Evidence

[1] Team 6 Project repository: https://github.com/soodujjwal/Team_6_Object_Oriented_Software

[2] Course material and instructor feedback

Evidence-key legend

- E1 backend/app/services/plaid_service.py
- E2 backend/app/api/plaid.py
- E3 frontend/src/components/PlaidConnection.tsx
- E4 frontend/src/domain/subscriptionInsights.ts
- E5 frontend/src/components/AppShell.tsx
- E6 frontend/tests/subscriptionInsights.test.mjs

AI-use disclosure draft: ChatGPT assisted with organizing, rewriting, and formatting this report from supplied documents and implementation summaries.